

KALITIM

NTP II - Hafta 4

NESNE TABANLI PROGRAMLAMADA KALITIM (INHERITANCE)

Bu sunum, Nesne Tabanlı Programlamanın (OOP) en temel prensiplerinden biri olan kalıtım mekanizmasını derinlemesine incelemektedir. Kalıtım, yazılım geliştirmede yeniden kullanılabilirliği (reusability) ve kod organizasyonunu artıran kritik bir araçtır. Hedefimiz, kalıtımın temel yapısını, kullanım senaryolarını ve gelişmiş özelliklerini (ezme, `super()` kullanımı) anlamaktır.

OOP'NİN TEMEL TAŞLARI VE KALITIMIN YERİ

Nesne Tabanlı Programlama dört ana prensip üzerine kuruludur:

1. Kapsülleme (Encapsulation): Verinin gizlenmesi ve korunması.
 2. Soyutlama (Abstraction): Gereksiz detayların gizlenmesi.
 3. Çok Biçimlilik (Polymorphism): Tek bir arayüzün birden çok şekil alabilmesi.
 4. Kalıtım (Inheritance): Sınıflar arasında özellik aktarımı.
- Kalıtım, sınıfların hiyerarşik bir ilişki kurmasını sağlayarak kodun modülerliğini artırır.

KALITIM NEDİR? TEMEL TANIM

Kalıtım, bir sınıfın (alt sınıf) başka bir sınıftan (ana sınıf) niteliklerini ve metotlarını miras alması sürecidir.

Bu mekanizma, yeni sınıflar oluşturulurken mevcut kodun yeniden kullanılmasını sağlar.

Böylece ortak özellikler ve davranışlar tek bir yerde tanımlanır ve tüm türetilmiş sınıflar tarafından erişilebilir hale gelir.



ANA SINIF VE ALT SINIF İLİŞKİSİ

Ana Sınıf (Parent Class / Base Class / Superclass):

Özellikleri miras bırakan sınıftır. Ortak ve genel davranışları tanımlar.

Alt Sınıf (Child Class / Derived Class / Subclass):

Ana sınıfın özelliklerini miras alan sınıftır. Ana sınıfın özelliklerini devralır ve kendine özgü yeni özellikler ekleyebilir veya mevcut özellikleri değiştirebilir.

MİRASIN TEMEL MANTIĞI: 'IS-A' İLİŞKİSİ

Kalıtım ilişkisi, daima 'is-a' (bir şeydir) ilişkisine dayanmalıdır.

Örnek: 'Öğrenci bir insandır' (Oğrenci is-a İnsan).

Bu ilişki, alt sınıfın mantıksal olarak ana sınıfın bir türü olduğunu garanti eder.

Eğer ilişki 'has-a' (sahiptir) şeklinde ise (Örnek: Araba bir Motora sahiptir), bu durumda kalıtım yerine Kompozisyon (Composition) kullanılmalıdır.



KALITIMIN TEMEL FAYDASI: KOD TEKRARINI ÖNLEME

Kalıtım, birden fazla sınıfta aynı şekilde tanımlanacak ortak niteliklerin ve metotların tek bir merkezde (ana sınıfta) toplanmasını sağlar. Bu, aynı kod bloğunu tekrar tekrar yazma gerekliliğini ortadan kaldırır. Kod tekrarı azaldıkça, yazılımın boyutu küçülür ve bakımı kolaylaşır.



Kalıtım, kodun mantıksal olarak hiyerarşik bir düzene girmesini sağlar. Özellikler ve davranışlar, ait oldukları soyutlama seviyesine göre ayrılır. Bu yapı, büyük projelerde yeni özellik eklemeyi veya hataları gidermeyi kolaylaştırır. Sürdürülebilir kod, daha az teknik borç birikimi anlamına gelir.

VAKA İNCELEMESİ: İNSAN SINIFI MODELİ

Bir 'Insan' ana sınıfı oluşturalım. Bu sınıf, tüm insanların ortak olarak sahip olduğu özellikleri taşımalıdır.

Ortak Nitelikler:

- Ad, Soyad, Yaş
- TC Kimlik Numarası

Ortak Metotlar:

- 'SelamVer()' (Adını söyleyerek selam verir)
- 'Yürü()'

Bu yapı, 'Insan'dan türeyecek tüm sınıflar için bir temel oluşturur.

ALT SINIFLARIN TÜRETİLMESİ: ÖĞRENCİ VE ÖĞRETMEN

'Insan' sınıfından türeyen 'Ogrenci' ve 'Ogretmen' sınıflarını inceleyelim.

'Ogrenci' sınıfı, 'Insan'ın tüm özelliklerini miras alır ve ek olarak:

- Öğrenci Numarası (Nitelik)
- 'DersCalis()' (Metot) gibi özelleşmiş üyeler ekler.

'Ogretmen' sınıfı ise:

- Branş (Nitelik)
- 'DersVer()' (Metot) gibi özelleşmiş üyeler ekler.

MİRAS ALINAN ÜYELERE ERİŞİM

Alt sınıflar, ana sınıftan miras aldıkları tüm 'public' ve 'protected' üyelere doğrudan erişebilirler.

Erişim Belirleyiciler:

- Public: Her yerden erişilebilir.
- Protected: Sadece miras alan alt sınıflardan erişilebilir.
- Private: Yalnızca tanımlandığı sınıftan erişilebilir, alt sınıflar dahi doğrudan erişemez.

Private üyeler miras alınır ancak doğrudan kullanılamaz; dolaylı olarak (genellikle public metotlar aracılığıyla) erişilebilir.

METOT EZME (METHOD OVERRIDING) PRENSİBİ

Metot ezme (override), alt sınıfın, ana sınıftan miras aldığı bir metodu kendine özgü bir şekilde yeniden tanımlamasıdır.
Alt sınıf, miras aldığı metodun imzasını (adı ve parametre listesini) korur, ancak içerideki işleyişini değiştirir.
Bu, Çok Biçimliliğin (Polymorphism) temelini oluşturan önemli bir özelliktir.

METOT EZMENİN KULLANIM ALANLARI

Örnek: 'Insan' sınıfının 'SelamVer()' metodu sadece 'Merhaba, ben [Adım]' desin.

'Ogretmen' sınıfı ise aynı 'SelamVer()' metodunu ezerek şunu söylesin:
'Merhaba, ben [Adım], [Branş] öğretmeniyim.'

Ezme, genel bir kuralı korurken türe özel detayları uygulamamızı sağlar.

METOT EZME KURALLARI

1. Metot imzası (Signature) Aynı Olmalıdır: Metodun adı ve parametre tipleri/sırası ana sınıftakiyle birebir uyuşmalıdır.
2. Erişim Kısıtlaması: Alt sınıftaki ezilmiş metodun erişim belirleyicisi, ana sınıftakinden daha kısıtlayıcı olamaz (Örn: public bir metot private yapılamaz).
3. Geri Dönüş Tipi: Geri dönüş tipi aynı olmalı veya covariant geri dönüş tipi (alt sınıf tipi) olmalıdır (Dil bağımlı).

SUPER() FONKSİYONU/ANAHTAR KELİMESİ

`super()` anahtar kelimesi, alt sınıfın, ana sınıfın metotlarına veya yapıcılarına (constructor) erişmesini sağlayan kritik bir yapıdır.

Bu, genellikle iki temel amaç için kullanılır:

1. Yapıcıyı çağırmak: Ana sınıfın başlatılmasını sağlamak.
2. Ezilmiş metodu çağırmak: Ezme işleminde ana sınıfın orijinal davranışını kullanmak ve üzerine ekleme yapmak.

SUPER() KULLANIMI: YAPICI METOTLAR

Bir alt sınıfın nesnesi oluşturulduğunda, genellikle önce ana sınıfın başlatılması gerekir.

Alt sınıfın yapıcısı içinde `super()` çağırılması, ana sınıfın yapıcısını çalıştırır ve miras alınan niteliklerin doğru değerlerle başlatılmasını sağlar.

Bu çağrı, alt sınıf yapıcısının en üst satırında yapılmalıdır (Dil gereksinimine göre değişebilir).

SUPER() KULLANIMI: EZİLMİŞ METOTLAR

Bir metot ezildiğinde, alt sınıf genellikle ana sınıfın orijinal davranışını tamamen silmek yerine onu genişletmek ister.

Örnek: Öğretmen sınıfı 'SelamVer()' metodunda, önce `super().SelamVer()` çağırısı yapar (böylece ana sınıfın 'Merhaba, ben [Adım]' çıktısı alınır), ardından öğretmenlik branşı bilgisini ekler.

Bu, kod tekrarını önlerken özelleştirmeye olanak tanır.

ÇOK BİÇİMLİLİK (POLYMORPHISM) VE KALITIM

Kalıtım ve Metot Ezme, Polimorfizmin hayata geçmesini sağlar. Polimorfizm, ana sınıf referansının, alt sınıfların nesnelerini tutabilmesi anlamına gelir.

Örnek: 'Insan' tipinde bir referans, hem 'Ogrenci' hem de 'Ogretmen' nesnelerine işaret edebilir.

Bu sayede, aynı metodu (Örnek: SelamVer()) çağırdığımızda, referansın işaret ettiği nesnenin gerçek tipine göre farklı davranış sergilenir (dinamik bağlama).



KALITIM TÜRLERİ – TEKLi KALITIM (SINGLE INHERITANCE)

Tekli kalıtım, bir alt sınıfın yalnızca tek bir ana sınıftan miras alması durumudur.

Bu, çoğu popüler nesne tabanlı programlama dilinde (Java, C#, Smalltalk) sınıflar için zorunlu kılınan modeldir.

Bu model, hiyerarşinin net ve anlaşılır olmasını sağlar.

KALITIM TÜRLERİ – ÇOK SEVİYELİ KALITIM

Çok seviyeli kalıtım, bir sınıfın başka bir alt sınıftan miras almasıdır ($A \rightarrow B \rightarrow C$).

C sınıfı, B'den miras alır. B sınıfı ise A'dan miras alır.

Bu durumda C, hem B'nin hem de A'nın özelliklerini miras alır.

Derin kalıtım zincirleri, kodun anlaşılabilirliğini zorlaştırabileceği için dikkatli kullanılmalıdır.

KALITIM TÜRLERİ – HİYERARŞİK KALITIM

Hiyerarşik kalıtım, tek bir ana sınıftan birden fazla alt sınıfın türemesi durumudur.

Örnek: 'Insan' ana sınıfından hem 'Öğrenci' hem de 'Öğretmen' sınıflarının türemesi hiyerarşik kalıtıma örnektir.

Bu, en yaygın ve faydalı kalıtım düzenlerinden biridir, çünkü ortak işlevsellik tek bir yerde tutulur.



ÇOKLU KALITIM (MULTIPLE INHERITANCE) KAVRAMI

Çoklu kalıtım, bir sınıfın birden fazla doğrudan ana sınıftan miras alabilmesi yeteneğidir.

Python ve C++ gibi dillerde bu özelliğe izin verilir, ancak Java ve C# gibi dillerde sınıflar için desteklenmez.

Temel nedeni, bu yapının 'Elmas Sorunu' (Diamond Problem) adı verilen karmaşık sorunlara yol açabilmesidir.

ÇOKLU KALITIMIN YOL AÇTIĞI SORUN: ELMAS PROBLEMİ

Elmas Problemi, A sınıfının B ve C sınıfları tarafından miras alınması, B ve C'nin ise D sınıfı tarafından miras alınması durumudur ($D \rightarrow B$, $D \rightarrow C$, $B \rightarrow A$, $C \rightarrow A$).

Eğer A sınıfında bir metot varsa, D sınıfı bu metodu B'den mi yoksa C'den mi miras almalıdır?

Bu belirsizlik, hangi metodun çağrılacağı konusunda derleyici için bir ikilem yaratır.

ARRAYÜZLERLE ÇÖZÜM: JAVA VE C# YAKLAŞIMI

Java ve C#, Elmas Sorununu aşmak ve yine de çoklu davranış (interface) kabiliyeti sağlamak için Arayüzler (Interfaces) kullanır.

Bir sınıf birden fazla arayüzden 'uygulama' (implement) yapabilir, ancak yalnızca tek bir sınıftan 'kalıtım' alabilir.

Arayüzler sadece metot imzalarını (sözleşmeleri) tanımladığından, metot gövdelerinin çatışma ihtimali ortadan kalkar.

SOYUT SINIFLAR (ABSTRACT CLASSES)

Soyut sınıflar, nesnesi doğrudan oluşturulamayan (örneklenemeyen) sınıflardır. Temel amaçları, alt sınıflar için bir şablon veya iskelet görevi görmektir.

Soyut sınıflar, hem normal metotlar (uygulaması olan) hem de soyut metotlar (uygulaması olmayan) içerebilir.

Eğer bir sınıf bir soyut metot içeriyorsa, sınıfın kendisi de soyut olmak zorundadır.



Soyut metot, ana sınıfta gövdesi tanımlanmamış (uygulaması olmayan) bir metottur.

Bu metotlar, alt sınıfların bu davranışı ****mutlaka**** kendi bünyelerinde tanımlamasını (ezmesini) zorunlu kılar.

Bu, tasarımcıların belirli işlevselliklerin türetilmiş sınıflarda bulunduğundan emin olmasını sağlar.



KONSTRÜKTÖRLER (YAPICILAR) VE KALITIM

Bir alt sınıf nesnesi oluşturulduğunda, yapıcı çağrı sırası her zaman kalıtım zincirinin en üstünden (ana sınıftan) başlar.

Alt sınıf yapıcısı çağrıldığında, alt sınıfın üyeleri başlatılmadan önce ana sınıfın yapıcısı çalıştırılmalıdır (genellikle `super()` ile).

Bu, miras alınan tüm parçaların düzgün bir şekilde başlatılmasını garantiler.

EZME VS AŞIRI YÜKLEME (OVERLOADING)

Metot Ezme (Overriding):

- Amaç: Ana sınıftan miras alınan bir metodu alt sınıfta yeniden tanımlamak.
- İlişki: Sınıflar arası (Kalıtım gerektirir).
- İmza: Aynı olmalıdır.

Metot Aşırı Yükleme (Overloading):

- Amaç: Aynı isimde birden fazla metot tanımlayarak farklı parametre tipleri veya sayıları ile çalışmak.
- İlişki: Aynı sınıf içinde veya kalıtım zincirinde.
- İmza: Farklı olmalıdır (Parametreler değişmelidir).

KALITIMIN KISITLARI: SIKI BAĞLANMA (TIGHT COUPLING)

Kalıtım, alt sınıf ile ana sınıf arasında çok sıkı bir bağ oluşturur. Alt sınıf, ana sınıfın iç yapısına (implementation details) bağımlı hale gelir. Eğer ana sınıfta yapılan bir değişiklik, alt sınıfların beklenmeyen şekilde bozulmasına neden olabilir (Kırılgan Taban Sınıf Sorunu – Fragile Base Class Problem).



KOMPOZİSYON (COMPOSITION) ALTERNATİFİ

Kompozisyon, kalıtıma alternatif olarak nesne ilişkilerini kurmak için kullanılır.

Kompozisyonda, bir sınıf başka bir sınıfın nesnesini kendi içinde bir nitelik olarak barındırır (Has-a ilişkisi).

Bu, sınıflar arasında daha gevşek bir bağ (loose coupling) sağlar ve esnekliği artırır.



PRENSİP: KALITIM YERİNE KOMPOZİSYONU TERCİH ET

Bu, modern yazılım tasarımında sıkça önerilen bir tasarım prensibidir. Kalıtım, sadece kesinlikle bir 'is-a' ilişkisi varsa kullanılmalıdır. Kompozisyon, sıkı bağlanma risklerini azaltır, kodun daha kolay test edilmesini sağlar ve çalışma zamanında davranışların dinamik olarak değiştirilmesine olanak tanır.



Kalıtım, alt sınıfı test ederken dolaylı olarak ana sınıfın davranışlarını da test etmeyi gerektirir.

Ana sınıfın karmaşıklığı, alt sınıfların birim testlerini karmaşıklıştırabilir. Kompozisyon kullanıldığında, alt sınıfların bağımlılıkları daha kolay alaylanabilir (mock) ve izole edilerek test edilebilir.



KULLANICI ARAYÜZÜ (GUI) BİLEŞENLERİNDE KALITIM

Kalıtım, görsel programlama kütüphanelerinde (Swing, AWT, .NET Forms) yaygın olarak kullanılır.

Temel Sınıf: 'Component' (Tüm görsel öğelerin ortak özelliklerini içerir: konum, boyut, görünürlük).

Türetilmiş Sınıflar: 'Button', 'TextBox', 'Label' gibi bileşenler 'Component'tan miras alır.

Bu yapı, tüm bileşenlerin temel davranışları paylaştığını garantilerken, her bileşenin kendine özgü işlevselliği eklemesini sağlar.

KALITIMI ENGELLEME: FINAL/SEALED SINIFLAR

Bazı durumlarda, bir sınıfın başka sınıflar tarafından miras alınmasını engellemek istenir.

Bunun nedenleri:

1. Güvenlik: Kritik sistem sınıflarının yanlışlıkla ezilmesini önlemek.
2. Performans: Derleyicinin optimizasyon yapabilmesi.
3. Tasarım Kararı: Sınıfın tamamlanmış ve genişletilmeye uygun olmadığına karar verilmesi.

Java'da 'final', C#'ta 'sealed' anahtar kelimeleri bu amaçla kullanılır.



KALITIM VE TASARIM KALIPLARI (DESIGN PATTERNS)

Kalıtım, birçok temel tasarım kalıbında merkezi bir rol oynar:

1. Şablon Metot Kalıbı (Template Method Pattern): Ana sınıf, algoritmanın iskeletini tanımlar (soyut metotlar kullanarak), alt sınıflar ise bu iskeletin belirli adımlarını uygular.
 2. Fabrika Metodu Kalıbı (Factory Method Pattern): Ana sınıf nesne yaratma sorumluluğunu alt sınıflara devreder.
- Bu kalıplar, kalıtımı esnek ve sürdürülebilir bir şekilde kullanmanın yollarını gösterir.

Kalıtım hiyerarşileri ne çok derin (çok seviyeli) ne de çok geniş (çok sayıda alt sınıf) olmalıdır.

Çok derin zincirler, bir metodun nereden geldiğini anlamayı zorlaştırır.

Çok geniş zincirler, ana sınıfın çok fazla sorumluluk taşıdığını (tek sorumluluk ilkesi ihlali) gösterebilir.

İdeal tasarım, mantıksal ve sığ bir hiyerarşiyi hedefler.

YAPISAL KALITIM (STRUCTURAL INHERITANCE)

Bu, bir alt sınıfın, ana sınıfın veri yapılarını (niteliklerini) miras alması ve kullanması anlamına gelir.

Örneğin, ana sınıfın kullandığı bir 'id' alanı, alt sınıflar tarafından otomatik olarak kullanılır.

Bu, verilerin tutarlılığını sağlamak için önemlidir, ancak sıkı bağlanmaya neden olabilir.



DAVRANIŞSAL KALITIM (BEHAVIORAL INHERITANCE)

Davranışsal kalıtım, ana sınıfta tanımlanan metotların (işlevselliklerin) alt sınıflar tarafından miras alınmasıdır.

Bu metotlar, alt sınıfların temel işlevlerini tanımlar ve metot ezme yoluyla özelleştirilebilir.

Kod tekrarını önlemede temel rol oynayan kalıtım türüdür.

ALT SINIFIN KURALI: LİSKOV YERİNE KOYMA İLKESİ (LSP)

LSP, SOLID prensiplerinin üçüncü harfidir ve kalıtımın doğru kullanımı için hayati öneme sahiptir.

İlke: 'Sistemde bir ana sınıf tipi (T) yerine, onun alt sınıfları (S) kullanıldığında, sistemin doğruluğu değişmemelidir.'

Basitçe: Alt sınıflar, ana sınıflarının yerine problemsiz bir şekilde geçebilmelidir.

Eğer bir alt sınıf, miras aldığı metodun davranışını kökten değiştiriyorsa (ezme yaparken tamamen farklı bir iş yapıyorsa), LSP ihlal edilmiş demektir.

LSP İHLALİ ÖRNEĞİ

Örnek: 'Kuş' ana sınıfında 'Uç()' metodu olsun.

'Penguen' alt sınıfı 'Kuş'tan miras alır ve 'Uç()' metodunu ezerek hata fırlatır veya hiçbir şey yapmaz.

Bu, 'Penguen' nesnesinin, 'Kuş' tipinde beklenti duyan kodu bozmasına neden olur.

Çözüm: Daha doğru bir soyutlama kurmak (Örnek: 'Canlı' -> 'Uçabilen' ve 'Uçamayan' alt arayüzler).

KALITIMDA YIKIMCI METOTLAR (DESTRUCTORS)

Yıkıcı metotlar, bir nesne bellekten silinmeden hemen önce çalıştırılır ve kaynakları serbest bırakmak için kullanılır (Örn: dosya bağlantılarını kapatmak).

Kalıtım zincirinde, yıkıcılar yapıcıların tam tersi sırada çalıştırılır: Önce alt sınıfın yıkıcısı, ardından ana sınıfın yıkıcısı çalışır.

Doğru sıralama, bellek sızıntılarını önlemek için önemlidir.



ARRAYÜZ KALITIMI (INTERFACE INHERITANCE)

Arayüzler teknik olarak metot uygulamalarını miras bırakmaz (Java 8/9 öncesi).

Ancak, bir arayüz başka bir arayüzden miras alabilir (genişletebilir).

Bu, yeni arayüzün, ebeveyn arayüzdeki tüm metot imzalarını devralması ve bu sözleşmeyi genişletmesi anlamına gelir.

Bu tür kalıtım, sadece metot imzalarını (tip kalıtımı) taşır, işlevselliği değil.

İNCE AYAR: MİRASA İZİN VERMEYEN METOTLAR

1. **Private Metotlar:** Alt sınıflar tarafından miras alınamaz veya görülemez. Bu metotlar, yalnızca ana sınıfın iç işleyişi için tasarlanmıştır.
2. **Final/Sealed Metotlar:** Miras alınır ancak alt sınıflar tarafından ezilemezler. Bu, ana sınıfın belirli bir davranışının tüm alt sınıflarda kesinlikle aynı kalmasını sağlar.



VERİ TİPİ DÖNÜŞÜMÜ (TYPE CASTING) VE KALITIM

Upcasting (Yukarı Dönüşüm):

Bir alt sınıf nesnesini güvenli bir şekilde ana sınıf referansına atamaktır (Örn: Öğrenci nesnesini İnsan referansına atamak).

Bu her zaman güvenlidir, çünkü alt sınıf, ana sınıfın tüm üyelerine sahiptir.

Downcasting (Aşağı Dönüşüm):

Bir ana sınıf referansını alt sınıf tipine döndürmektir.

Bu işlem potansiyel olarak güvensizdir ve çalışma zamanı hatasına neden olabilir (eğer referans, istenen alt sınıf nesnesini tutmuyorsa).

MİRAS VE KOLEKSİYONLAR

Kalıtım, koleksiyonları yönetirken güçlü bir esneklik sağlar.
Bir 'Insan' listesi (array veya list) tanımlandığında, bu liste hem 'Ogrenci' hem de 'Ogretmen' nesnelerini barındırabilir.
Bu, Polimorfizm sayesinde, farklı alt sınıf nesnelere aynı ana sınıf arayüzü üzerinden erişme ve işlem yapma imkanı verir.

MİRAS ALINAN NİTELİKLERDEKİ GÖLGELEME (HIDING)

Gölgeleme, alt sınıfın, ana sınıfta tanımlanmış aynı isme sahip bir nitelik (field) tanımlamasıdır.

Bu durumda, alt sınıfın nesnesi aslında iki farklı niteliğe sahiptir (biri kendi, diğeri miras alınan).

Ezmenin (metotlar için) aksine, gölgeleme tehlikelidir ve genellikle kafa karışıklığına yol açar. İyi bir tasarımda kaçınılmalıdır.

Nesne ilişkisel Eşleme (ORM) araçları, kalıtım hiyerarşilerini veritabanı tablolarına dönüştürmek için farklı stratejiler kullanır:

1. Her Sınıf İçin Bir Tablo (Table Per Class):
 2. Tüm Hiyerarşi İçin Tek Tablo (Single Table Inheritance):
 3. Somut Sınıflar İçin Tablo (Table Per Concrete Class):
- Bu stratejiler, OOP kalıtımının veritabanı yapısına nasıl taşınacağını belirler.

KALITIMDA UZAYLAMA (NAMESPACEING) VE PAKETLEME

Bazı dillerde (Örn: Java), farklı paketlerdeki sınıflar arasında kalıtım kurulurken erişim belirleyicilere özellikle dikkat edilmelidir. 'Default' veya 'Package-Private' erişime sahip üyeler, farklı bir paketteki alt sınıf tarafından miras alınsalar bile erişilemezler. Global erişime izin vermek için 'public' veya 'protected' kullanılmalıdır.

MİRAS ALINAN KODUN DEĞİŞTİRİLMESİ

Ana sınıfta yapılan en ufak bir değişiklik bile (Örn: bir metodun imzasını değiştirmek, bir niteliğin adını değiştirmek), o sınıftan türemiş tüm alt sınıfların kırılmasına yol açabilir.

Bu, büyük ve uzun ömürlü projelerde kalıtımın dikkatli kullanılmasını gerektiren en büyük risk faktörüdür.

Ana sınıfların kararlı ve iyi belgelenmiş olması esastır.



DELEGASYON (DELEGATION) VE KALITIM

Delegasyon, bir nesnenin işi kendi yapmak yerine, başka bir nesnenin ilgili metotlarını çağırmasıdır.

Kalıtım (özellik aktarımı) yerine, delegasyon (sorumluluk aktarımı) kullanarak gevşek bağlanma sağlanabilir.

Örnek: Bir 'Yazıcı' sınıfı, yazdırma işlemini kendi yapmaz, bu işi 'YazdırmaMekanizması' nesnesine delege eder.



MIXIN'LER: ÇOKLU KALITIMA FONKSİYONEL YAKLAŞIM

Python veya Ruby gibi dillerde kullanılan Mixin'ler, bir sınıfa yeniden kullanılabilir metot setleri eklemek için kullanılır.

Mixin'ler, genellikle durum (state/nitelik) taşımazlar, yalnızca davranış (metot) sağlarlar.

Bu, çoklu kalıtımın getirdiği yapısal karmaşayı yaşamadan, sınıflara çoklu işlevsellik katmanın temiz bir yoludur.



Bir 'Mesaj' ana sınıfı, genel metotları (Örn: 'Gonder()') tanımlar. 'TurkceMesaj', 'IngilizceMesaj' gibi alt sınıflar, mesajın diline özgü içerik oluşturma metotlarını ezebilir veya yeni dil odaklı nitelikler ekleyebilir. Bu, uluslararasılaşma (i18n) çalışmalarında hiyerarşik yapı kurmak için idealdir.



KALITIMDA PERFORMANS DEĞERLENDİRMESİ

Kalıtımın kendisi genellikle performansta büyük bir darboğaz yaratmaz. Ancak, dinamik bağlama (metot ezme/polimorfizm) kullanıldığında, metot çağrısının hangi sınıfa ait olduğunun çalışma zamanında belirlenmesi gerekir (dynamic dispatch). Bu, derleyici optimizasyonlarını (Örn: inline etme) kısıtlayabilir ve saf sanal olmayan metot çağrılarından biraz daha yavaş olabilir, ancak modern VM'lerde fark ihmal edilebilir düzeydedir.



Eğer bir ana sınıf, yönetilmesi gereken harici kaynaklara (dosyalar, ağ bağlantıları) sahipse, alt sınıfların da bu kaynakları düzgün bir şekilde serbest bırakması (cleanup) gerekir.

Bu, yapıcı ve yıkıcı (destructor) zincirlerinin ve 'try-finally' veya 'using' bloklarının doğru kullanılmasını zorunlu kılar.

Aksi takdirde, alt sınıflar ana sınıfın kaynaklarını açık bırakarak sızıntılara neden olabilir.



KALITIMIN KOD KALİTESİNE ETKİSİ

Doğru kullanıldığında:

- 1. Yeniden kullanılabilirlik ve Modülerlik artar.**
- 2. Kod hacmi azalır.**
- 3. Sınıf hiyerarşisi mantıksal bir düzen kazanır.**

Yanlış kullanıldığında (LSP ihlali veya gereksiz derinlik):

- 1. Kodun esnekliği azalır (Sıkı Bağlanma).**
- 2. Sistem bakımı zorlaşır (Kırılgan Taban Sınıfı).**



GENEL ÖZET: KALITIMIN TEMEL İŞLEVLERİ

Kalıtım, temel olarak iki işlevi yerine getirir:

1. **Özellik Aktarımı:** Ana sınıftan nitelik ve metotları alt sınıflara geçirerek kod tekrarını önler.
2. **Polimorfizm için Temel Sağlama:** Metot ezme ve ana sınıf referansları aracılığıyla esnek davranışlar oluşturulmasını sağlar.

SONUÇ: İYİ TASARIM İÇİN İPUÇLARI

1. Sadece 'is-a' ilişkisi varsa kalıtımı kullanın.
2. Kalıtım hiyerarşilerini sıkı tutun (2-3 seviyeden daha derin olmamalı).
3. Ana sınıfı (Superclass) çok genel tutmaktan kaçının (çok fazla sorumluluk yüklemeyin).
4. Mümkünse, somut sınıflardan kalıtım yerine arayüzleri uygulamayı veya soyut sınıfları kullanmayı tercih edin.
5. Çoğu durumda, Kompozisyonu Kalıtıma tercih etmeyi değerlendirin (Has-a ilişkileri).

NESNE TABANLI PROGRAMLAMADA KALITIM (INHERITANCE)

Bu sunum, Nesne Tabanlı Programlamanın (OOP) en temel prensiplerinden biri olan kalıtım mekanizmasını derinlemesine incelemektedir. Kalıtım, yazılım geliştirmede yeniden kullanılabilirliği (reusability) ve kod organizasyonunu artıran kritik bir araçtır. Hedefimiz, kalıtımın temel yapısını, kullanım senaryolarını ve gelişmiş özelliklerini (ezme, `super()` kullanımı) anlamaktır.

OOP'NİN TEMEL TAŞLARI VE KALITIMIN YERİ

Nesne Tabanlı Programlama dört ana prensip üzerine kuruludur:

1. Kapsülleme (Encapsulation): Verinin gizlenmesi ve korunması.
 2. Soyutlama (Abstraction): Gereksiz detayların gizlenmesi.
 3. Çok Biçimlilik (Polymorphism): Tek bir arayüzün birden çok şekil alabilmesi.
 4. Kalıtım (Inheritance): Sınıflar arasında özellik aktarımı.
- Kalıtım, sınıfların hiyerarşik bir ilişki kurmasını sağlayarak kodun modülerliğini artırır.



KALITIM NEDİR? TEMEL TANIM

Kalıtım, bir sınıfın (alt sınıf) başka bir sınıftan (ana sınıf) niteliklerini ve metotlarını miras alması sürecidir.

Bu mekanizma, yeni sınıflar oluşturulurken mevcut kodun yeniden kullanılmasını sağlar.

Böylece ortak özellikler ve davranışlar tek bir yerde tanımlanır ve tüm türetilmiş sınıflar tarafından erişilebilir hale gelir.

ANA SINIF VE ALT SINIF İLİŞKİSİ

Ana Sınıf (Parent Class / Base Class / Superclass):

Özellikleri miras bırakan sınıftır. Ortak ve genel davranışları tanımlar.

Alt Sınıf (Child Class / Derived Class / Subclass):

Ana sınıfın özelliklerini miras alan sınıftır. Ana sınıfın özelliklerini devralır ve kendine özgü yeni özellikler ekleyebilir veya mevcut özellikleri değiştirebilir.



MİRASIN TEMEL MANTIĞI: 'IS-A' İLİŞKİSİ

Kalıtım ilişkisi, daima 'is-a' (bir şeydir) ilişkisine dayanmalıdır.

Örnek: 'Öğrenci bir insandır' (Oğrenci is-a İnsan).

Bu ilişki, alt sınıfın mantıksal olarak ana sınıfın bir türü olduğunu garanti eder.

Eğer ilişki 'has-a' (sahiptir) şeklinde ise (Örnek: Araba bir Motora sahiptir), bu durumda kalıtım yerine Kompozisyon (Composition) kullanılmalıdır.



KALITIMIN TEMEL FAYDASI: KOD TEKRARINI ÖNLEME

Kalıtım, birden fazla sınıfta aynı şekilde tanımlanacak ortak niteliklerin ve metotların tek bir merkezde (ana sınıfta) toplanmasını sağlar. Bu, aynı kod bloğunu tekrar tekrar yazma gerekliliğini ortadan kaldırır. Kod tekrarı azaldıkça, yazılımın boyutu küçülür ve bakımı kolaylaşır.

Kalıtım, kodun mantıksal olarak hiyerarşik bir düzene girmesini sağlar. Özellikler ve davranışlar, ait oldukları soyutlama seviyesine göre ayrılır. Bu yapı, büyük projelerde yeni özellik eklemeyi veya hataları gidermeyi kolaylaştırır. Sürdürülebilir kod, daha az teknik borç birikimi anlamına gelir.



VAKA İNCELEMESİ: İNSAN SINIFI MODELİ

Bir 'Insan' ana sınıfı oluşturalım. Bu sınıf, tüm insanların ortak olarak sahip olduğu özellikleri taşımalıdır.

Ortak Nitelikler:

- Ad, Soyad, Yaş
- TC Kimlik Numarası

Ortak Metotlar:

- 'SelamVer()' (Adını söyleyerek selam verir)
- 'Yürü()'

Bu yapı, 'Insan'dan türeyecek tüm sınıflar için bir temel oluşturur.

ALT SINIFLARIN TÜRETİLMESİ: ÖĞRENCİ VE ÖĞRETMEN

'Insan' sınıfından türeyen 'Ogrenci' ve 'Ogretmen' sınıflarını inceleyelim.

'Ogrenci' sınıfı, 'Insan'ın tüm özelliklerini miras alır ve ek olarak:

- Öğrenci Numarası (Nitelik)
- 'DersCalis()' (Metot) gibi özelleşmiş üyeler ekler.

'Ogretmen' sınıfı ise:

- Branş (Nitelik)
- 'DersVer()' (Metot) gibi özelleşmiş üyeler ekler.

MİRAS ALINAN ÜYELERE ERİŞİM

Alt sınıflar, ana sınıftan miras aldıkları tüm 'public' ve 'protected' üyelere doğrudan erişebilirler.

Erişim Belirleyiciler:

- Public: Her yerden erişilebilir.
- Protected: Sadece miras alan alt sınıflardan erişilebilir.
- Private: Yalnızca tanımlandığı sınıftan erişilebilir, alt sınıflar dahi doğrudan erişemez.

Private üyeler miras alınır ancak doğrudan kullanılamaz; dolaylı olarak (genellikle public metotlar aracılığıyla) erişilebilir.



METOT EZME (METHOD OVERRIDING) PRENSİBİ

Metot ezme (override), alt sınıfın, ana sınıftan miras aldığı bir metodu kendine özgü bir şekilde yeniden tanımlamasıdır.
Alt sınıf, miras aldığı metodun imzasını (adı ve parametre listesini) korur, ancak içerideki işleyişini değiştirir.
Bu, Çok Biçimliliğin (Polymorphism) temelini oluşturan önemli bir özelliktir.

METOT EZMENİN KULLANIM ALANLARI

Örnek: 'Insan' sınıfının 'SelamVer()' metodu sadece 'Merhaba, ben [Adım]' desin.

'Ogretmen' sınıfı ise aynı 'SelamVer()' metodunu ezerek şunu söylesin:
'Merhaba, ben [Adım], [Branş] öğretmeniyim.'

Ezme, genel bir kuralı korurken türe özel detayları uygulamamızı sağlar.



METOT EZME KURALLARI

1. Metot imzası (Signature) Aynı Olmalıdır: Metodun adı ve parametre tipleri/sırası ana sınıftakiyle birebir uyuşmalıdır.
2. Erişim Kısıtlaması: Alt sınıftaki ezilmiş metodun erişim belirleyicisi, ana sınıftakinden daha kısıtlayıcı olamaz (Örn: public bir metot private yapılamaz).
3. Geri Dönüş Tipi: Geri dönüş tipi aynı olmalı veya covariant geri dönüş tipi (alt sınıf tipi) olmalıdır (Dil bağımlı).

SUPER() FONKSİYONU/ANAHTAR KELİMESİ

`super()` anahtar kelimesi, alt sınıfın, ana sınıfın metotlarına veya yapıcılarına (constructor) erişmesini sağlayan kritik bir yapıdır.

Bu, genellikle iki temel amaç için kullanılır:

1. Yapıcıyı çağırmak: Ana sınıfın başlatılmasını sağlamak.
2. Ezilmiş metodu çağırmak: Ezme işleminde ana sınıfın orijinal davranışını kullanmak ve üzerine ekleme yapmak.

SUPER() KULLANIMI: YAPICI METOTLAR

Bir alt sınıfın nesnesi oluşturulduğunda, genellikle önce ana sınıfın başlatılması gerekir.

Alt sınıfın yapıcısı içinde `super()` çağırılması, ana sınıfın yapıcısını çalıştırır ve miras alınan niteliklerin doğru değerlerle başlatılmasını sağlar.

Bu çağrı, alt sınıf yapıcısının en üst satırında yapılmalıdır (Dil gereksinimine göre değişebilir).



SUPER() KULLANIMI: EZİLMİŞ METOTLAR

Bir metot ezildiğinde, alt sınıf genellikle ana sınıfın orijinal davranışını tamamen silmek yerine onu genişletmek ister.

Örnek: Öğretmen sınıfı 'SelamVer()' metodunda, önce `super().SelamVer()` çağırısı yapar (böylece ana sınıfın 'Merhaba, ben [Adım]' çıktısı alınır), ardından öğretmenlik branşı bilgisini ekler.

Bu, kod tekrarını önlerken özelleştirmeye olanak tanır.

ÇOK BİÇİMLİLİK (POLYMORPHISM) VE KALITIM

Kalıtım ve Metot Ezme, Polimorfizmin hayata geçmesini sağlar. Polimorfizm, ana sınıf referansının, alt sınıfların nesnelerini tutabilmesi anlamına gelir.

Örnek: 'Insan' tipinde bir referans, hem 'Ogrenci' hem de 'Ogretmen' nesnelerine işaret edebilir.

Bu sayede, aynı metodu (Örnek: SelamVer()) çağırdığımızda, referansın işaret ettiği nesnenin gerçek tipine göre farklı davranış sergilenir (dinamik bağlama).



KALITIM TÜRLERİ – TEKİLİ KALITIM (SINGLE INHERITANCE)

Tekli kalıtım, bir alt sınıfın yalnızca tek bir ana sınıftan miras alması durumudur.

Bu, çoğu popüler nesne tabanlı programlama dilinde (Java, C#, Smalltalk) sınıflar için zorunlu kılınan modeldir.

Bu model, hiyerarşinin net ve anlaşılır olmasını sağlar.

KALITIM TÜRLERİ – ÇOK SEVİYELİ KALITIM

Çok seviyeli kalıtım, bir sınıfın başka bir alt sınıftan miras almasıdır ($A \rightarrow B \rightarrow C$).

C sınıfı, B'den miras alır. B sınıfı ise A'dan miras alır.

Bu durumda C, hem B'nin hem de A'nın özelliklerini miras alır.

Derin kalıtım zincirleri, kodun anlaşılabilirliğini zorlaştırabileceği için dikkatli kullanılmalıdır.

KALITIM TÜRLERİ – HİYERARŞİK KALITIM

Hiyerarşik kalıtım, tek bir ana sınıftan birden fazla alt sınıfın türemesi durumudur.

Örnek: 'Insan' ana sınıfından hem 'Öğrenci' hem de 'Öğretmen' sınıflarının türemesi hiyerarşik kalıtıma örnektir.

Bu, en yaygın ve faydalı kalıtım düzenlerinden biridir, çünkü ortak işlevsellik tek bir yerde tutulur.



ÇOKLU KALITIM (MULTIPLE INHERITANCE) KAVRAMI

Çoklu kalıtım, bir sınıfın birden fazla doğrudan ana sınıftan miras alabilmesi yeteneğidir.

Python ve C++ gibi dillerde bu özelliğe izin verilir, ancak Java ve C# gibi dillerde sınıflar için desteklenmez.

Temel nedeni, bu yapının 'Elmas Sorunu' (Diamond Problem) adı verilen karmaşık sorunlara yol açabilmesidir.



ÇOKLU KALITIMIN YOL AÇTIĞI SORUN: ELMAS PROBLEMİ

Elmas Problemi, A sınıfının B ve C sınıfları tarafından miras alınması, B ve C'nin ise D sınıfı tarafından miras alınması durumudur ($D \rightarrow B$, $D \rightarrow C$, $B \rightarrow A$, $C \rightarrow A$).

Eğer A sınıfında bir metod varsa, D sınıfı bu metodu B'den mi yoksa C'den mi miras almalıdır?

Bu belirsizlik, hangi metodun çağrılacağı konusunda derleyici için bir ikilem yaratır.



ARAYÜZLERLE ÇÖZÜM: JAVA VE C# YAKLAŞIMI

Java ve C#, Elmas Sorununu aşmak ve yine de çoklu davranış (interface) kabiliyeti sağlamak için Arayüzler (Interfaces) kullanır.

Bir sınıf birden fazla arayüzden 'uygulama' (implement) yapabilir, ancak yalnızca tek bir sınıftan 'kalıtım' alabilir.

Arayüzler sadece metot imzalarını (sözleşmeleri) tanımladığından, metot gövdelerinin çatışma ihtimali ortadan kalkar.



SOYUT SINIFLAR (ABSTRACT CLASSES)

Soyut sınıflar, nesnesi doğrudan oluşturulamayan (örneklenemeyen) sınıflardır. Temel amaçları, alt sınıflar için bir şablon veya iskelet görevi görmektir.

Soyut sınıflar, hem normal metotlar (uygulaması olan) hem de soyut metotlar (uygulaması olmayan) içerebilir.

Eğer bir sınıf bir soyut metot içeriyorsa, sınıfın kendisi de soyut olmak zorundadır.



Soyut metot, ana sınıfta gövdesi tanımlanmamış (uygulaması olmayan) bir metottur.

Bu metotlar, alt sınıfların bu davranışı ****mutlaka**** kendi bünyelerinde tanımlamasını (ezmesini) zorunlu kılar.

Bu, tasarımcıların belirli işlevselliklerin türetilmiş sınıflarda bulunduğundan emin olmasını sağlar.

KONSTRÜKTÖRLER (YAPICILAR) VE KALITIM

Bir alt sınıf nesnesi oluşturulduğunda, yapıcı çağrı sırası her zaman kalıtım zincirinin en üstünden (ana sınıftan) başlar.

Alt sınıf yapıcısı çağrıldığında, alt sınıfın üyeleri başlatılmadan önce ana sınıfın yapıcısı çalıştırılmalıdır (genellikle `super()` ile).

Bu, miras alınan tüm parçaların düzgün bir şekilde başlatılmasını garantiler.

EZME VS AŞIRI YÜKLEME (OVERLOADING)

Metot Ezme (Overriding):

- Amaç: Ana sınıftan miras alınan bir metodu alt sınıfta yeniden tanımlamak.
- İlişki: Sınıflar arası (Kalıtım gerektirir).
- İmza: Aynı olmalıdır.

Metot Aşırı Yükleme (Overloading):

- Amaç: Aynı isimde birden fazla metot tanımlayarak farklı parametre tipleri veya sayıları ile çalışmak.
- İlişki: Aynı sınıf içinde veya kalıtım zincirinde.
- İmza: Farklı olmalıdır (Parametreler değişmelidir).

KALITIMIN KISITLARI: SIKI BAĞLANMA (TIGHT COUPLING)

Kalıtım, alt sınıf ile ana sınıf arasında çok sıkı bir bağ oluşturur. Alt sınıf, ana sınıfın iç yapısına (implementation details) bağımlı hale gelir. Eğer ana sınıfta yapılan bir değişiklik, alt sınıfların beklenmeyen şekilde bozulmasına neden olabilir (Kırılgan Taban Sınıf Sorunu – Fragile Base Class Problem).



KOMPOZİSYON (COMPOSITION) ALTERNATİFİ

Kompozisyon, kalıtıma alternatif olarak nesne ilişkilerini kurmak için kullanılır.

Kompozisyonda, bir sınıf başka bir sınıfın nesnesini kendi içinde bir nitelik olarak barındırır (Has-a ilişkisi).

Bu, sınıflar arasında daha gevşek bir bağ (loose coupling) sağlar ve esnekliği artırır.



PRENSİP: KALITIM YERİNE KOMPOZİSYONU TERCİH ET

Bu, modern yazılım tasarımında sıkça önerilen bir tasarım prensibidir. Kalıtım, sadece kesinlikle bir 'is-a' ilişkisi varsa kullanılmalıdır. Kompozisyon, sıkı bağlanma risklerini azaltır, kodun daha kolay test edilmesini sağlar ve çalışma zamanında davranışların dinamik olarak değiştirilmesine olanak tanır.



Kalıtım, alt sınıfı test ederken dolaylı olarak ana sınıfın davranışlarını da test etmeyi gerektirir.

Ana sınıfın karmaşıklığı, alt sınıfların birim testlerini karmaşıktırabilir. Kompozisyon kullanıldığında, alt sınıfların bağımlılıkları daha kolay alaylanabilir (mock) ve izole edilerek test edilebilir.



KULLANICI ARAYÜZÜ (GUI) BİLEŞENLERİNDE KALITIM

Kalıtım, görsel programlama kütüphanelerinde (Swing, AWT, .NET Forms) yaygın olarak kullanılır.

Temel Sınıf: 'Component' (Tüm görsel öğelerin ortak özelliklerini içerir: konum, boyut, görünürlük).

Türetilmiş Sınıflar: 'Button', 'TextBox', 'Label' gibi bileşenler 'Component'tan miras alır.

Bu yapı, tüm bileşenlerin temel davranışları paylaştığını garantilerken, her bileşenin kendine özgü işlevselliği eklemesini sağlar.

KALITIMI ENGELLEME: FINAL/SEALED SINIFLAR

Bazı durumlarda, bir sınıfın başka sınıflar tarafından miras alınmasını engellemek istenir.

Bunun nedenleri:

1. Güvenlik: Kritik sistem sınıflarının yanlışlıkla ezilmesini önlemek.
2. Performans: Derleyicinin optimizasyon yapabilmesi.
3. Tasarım Kararı: Sınıfın tamamlanmış ve genişletilmeye uygun olmadığına karar verilmesi.

Java'da 'final', C#'ta 'sealed' anahtar kelimeleri bu amaçla kullanılır.

KALITIM VE TASARIM KALIPLARI (DESIGN PATTERNS)

Kalıtım, birçok temel tasarım kalıbında merkezi bir rol oynar:

1. Şablon Metot Kalıbı (Template Method Pattern): Ana sınıf, algoritmanın iskeletini tanımlar (soyut metotlar kullanarak), alt sınıflar ise bu iskeletin belirli adımlarını uygular.
 2. Fabrika Metodu Kalıbı (Factory Method Pattern): Ana sınıf nesne yaratma sorumluluğunu alt sınıflara devreder.
- Bu kalıplar, kalıtımı esnek ve sürdürülebilir bir şekilde kullanmanın yollarını gösterir.



Kalıtım hiyerarşileri ne çok derin (çok seviyeli) ne de çok geniş (çok sayıda alt sınıf) olmalıdır.

Çok derin zincirler, bir metodun nereden geldiğini anlamayı zorlaştırır.

Çok geniş zincirler, ana sınıfın çok fazla sorumluluk taşıdığını (tek sorumluluk ilkesi ihlali) gösterebilir.

İdeal tasarım, mantıksal ve sığ bir hiyerarşiyi hedefler.

YAPISAL KALITIM (STRUCTURAL INHERITANCE)

Bu, bir alt sınıfın, ana sınıfın veri yapılarını (niteliklerini) miras alması ve kullanması anlamına gelir.

Örneğin, ana sınıfın kullandığı bir 'id' alanı, alt sınıflar tarafından otomatik olarak kullanılır.

Bu, verilerin tutarlılığını sağlamak için önemlidir, ancak sıkı bağlanmaya neden olabilir.



DAVRANIŞSAL KALITIM (BEHAVIORAL INHERITANCE)

Davranışsal kalıtım, ana sınıfta tanımlanan metotların (işlevselliklerin) alt sınıflar tarafından miras alınmasıdır.

Bu metotlar, alt sınıfların temel işlevlerini tanımlar ve metot ezme yoluyla özelleştirilebilir.

Kod tekrarını önlemede temel rol oynayan kalıtım türüdür.



ALT SINIFIN KURALI: LİSKOV YERİNE KOYMA İLKESİ (LSP)

LSP, SOLID prensiplerinin üçüncü harfidir ve kalıtımın doğru kullanımı için hayati öneme sahiptir.

İlke: 'Sistemde bir ana sınıf tipi (T) yerine, onun alt sınıfları (S) kullanıldığında, sistemin doğruluğu değişmemelidir.'

Basitçe: Alt sınıflar, ana sınıflarının yerine problemsiz bir şekilde geçebilmelidir.

Eğer bir alt sınıf, miras aldığı metodun davranışını kökten değiştiriyorsa (ezme yaparken tamamen farklı bir iş yapıyorsa), LSP ihlal edilmiş demektir.

LSP İHLALİ ÖRNEĞİ

Örnek: 'Kuş' ana sınıfında 'Uç()' metodu olsun.

'Penguen' alt sınıfı 'Kuş'tan miras alır ve 'Uç()' metodunu ezerek hata fırlatır veya hiçbir şey yapmaz.

Bu, 'Penguen' nesnesinin, 'Kuş' tipinde beklenti duyan kodu bozmasına neden olur.

Çözüm: Daha doğru bir soyutlama kurmak (Örnek: 'Canlı' -> 'Uçabilen' ve 'Uçamayan' alt arayüzler).



KALITIMDA YIKIMCI METOTLAR (DESTRUCTORS)

Yıkıcı metotlar, bir nesne bellekten silinmeden hemen önce çalıştırılır ve kaynakları serbest bırakmak için kullanılır (Örn: dosya bağlantılarını kapatmak).

Kalıtım zincirinde, yıkıcılar yapıcıların tam tersi sırada çalıştırılır: Önce alt sınıfın yıkıcısı, ardından ana sınıfın yıkıcısı çalışır.

Doğru sıralama, bellek sızıntılarını önlemek için önemlidir.



ARRAYÜZ KALITIMI (INTERFACE INHERITANCE)

Arayüzler teknik olarak metot uygulamalarını miras bırakmaz (Java 8/9 öncesi).

Ancak, bir arayüz başka bir arayüzden miras alabilir (genişletebilir).

Bu, yeni arayüzün, ebeveyn arayüzdeki tüm metot imzalarını devralması ve bu sözleşmeyi genişletmesi anlamına gelir.

Bu tür kalıtım, sadece metot imzalarını (tip kalıtımı) taşır, işlevselliği değil.

İNCE AYAR: MİRASA İZİN VERMEYEN METOTLAR

1. **Private Metotlar:** Alt sınıflar tarafından miras alınamaz veya görülemez. Bu metotlar, yalnızca ana sınıfın iç işleyişi için tasarlanmıştır.
2. **Final/Sealed Metotlar:** Miras alınır ancak alt sınıflar tarafından ezilemezler. Bu, ana sınıfın belirli bir davranışının tüm alt sınıflarda kesinlikle aynı kalmasını sağlar.



VERİ TİPİ DÖNÜŞÜMÜ (TYPE CASTING) VE KALITIM

Upcasting (Yukarı Dönüşüm):

Bir alt sınıf nesnesini güvenli bir şekilde ana sınıf referansına atamaktır (Örn: Öğrenci nesnesini İnsan referansına atamak).

Bu her zaman güvenlidir, çünkü alt sınıf, ana sınıfın tüm üyelerine sahiptir.

Downcasting (Aşağı Dönüşüm):

Bir ana sınıf referansını alt sınıf tipine döndürmektir.

Bu işlem potansiyel olarak güvensizdir ve çalışma zamanı hatasına neden olabilir (eğer referans, istenen alt sınıf nesnesini tutmuyorsa).

MİRAS VE KOLEKSİYONLAR

Kalıtım, koleksiyonları yönetirken güçlü bir esneklik sağlar.
Bir 'Insan' listesi (array veya list) tanımlandığında, bu liste hem 'Ogrenci' hem de 'Ogretmen' nesnelerini barındırabilir.
Bu, Polimorfizm sayesinde, farklı alt sınıf nesnelere aynı ana sınıf arayüzü üzerinden erişme ve işlem yapma imkanı verir.

MİRAS ALINAN NİTELİKLERDEKİ GÖLGELEME (HIDING)

Gölgeleme, alt sınıfın, ana sınıfta tanımlanmış aynı isme sahip bir nitelik (field) tanımlamasıdır.

Bu durumda, alt sınıfın nesnesi aslında iki farklı niteliğe sahiptir (biri kendi, diğeri miras alınan).

Ezmenin (metotlar için) aksine, gölgeleme tehlikelidir ve genellikle kafa karışıklığına yol açar. İyi bir tasarımda kaçınılmalıdır.

Nesne ilişkisel Eşleme (ORM) araçları, kalıtım hiyerarşilerini veritabanı tablolarına dönüştürmek için farklı stratejiler kullanır:

1. Her Sınıf İçin Bir Tablo (Table Per Class):
 2. Tüm Hiyerarşi İçin Tek Tablo (Single Table Inheritance):
 3. Somut Sınıflar İçin Tablo (Table Per Concrete Class):
- Bu stratejiler, OOP kalıtımının veritabanı yapısına nasıl taşınacağını belirler.

KALITIMDA UZAYLAMA (NAMESPACEING) VE PAKETLEME

Bazı dillerde (Örn: Java), farklı paketlerdeki sınıflar arasında kalıtım kurulurken erişim belirleyicilere özellikle dikkat edilmelidir. 'Default' veya 'Package-Private' erişime sahip üyeler, farklı bir paketteki alt sınıf tarafından miras alınsalar bile erişilemezler. Global erişime izin vermek için 'public' veya 'protected' kullanılmalıdır.

MİRAS ALINAN KODUN DEĞİŞTİRİLMESİ

Ana sınıfta yapılan en ufak bir değişiklik bile (Örn: bir metodun imzasını değiştirmek, bir niteliğin adını değiştirmek), o sınıftan türemiş tüm alt sınıfların kırılmasına yol açabilir.

Bu, büyük ve uzun ömürlü projelerde kalıtımın dikkatli kullanılmasını gerektiren en büyük risk faktörüdür.

Ana sınıfların kararlı ve iyi belgelenmiş olması esastır.



DELEGASYON (DELEGATION) VE KALITIM

Delegasyon, bir nesnenin işi kendi yapmak yerine, başka bir nesnenin ilgili metotlarını çağırmasıdır.

Kalıtım (özellik aktarımı) yerine, delegasyon (sorumluluk aktarımı) kullanarak gevşek bağlanma sağlanabilir.

Örnek: Bir 'Yazıcı' sınıfı, yazdırma işlemini kendi yapmaz, bu işi 'YazdırmaMekanizması' nesnesine delege eder.

MIXIN'LER: ÇOKLU KALITIMA FONKSİYONEL YAKLAŞIM

Python veya Ruby gibi dillerde kullanılan Mixin'ler, bir sınıfa yeniden kullanılabilir metot setleri eklemek için kullanılır.

Mixin'ler, genellikle durum (state/nitelik) taşımazlar, yalnızca davranış (metot) sağlarlar.

Bu, çoklu kalıtımın getirdiği yapısal karmaşayı yaşamadan, sınıflara çoklu işlevsellik katmanın temiz bir yoludur.



Bir 'Mesaj' ana sınıfı, genel metotları (Örn: 'Gonder()') tanımlar. 'TurkceMesaj', 'IngilizceMesaj' gibi alt sınıflar, mesajın diline özgü içerik oluşturma metotlarını ezebilir veya yeni dil odaklı nitelikler ekleyebilir. Bu, uluslararasılaşma (i18n) çalışmalarında hiyerarşik yapı kurmak için idealdir.



Kalıtımın kendisi genellikle performansta büyük bir darboğaz yaratmaz. Ancak, dinamik bağlama (metot ezme/polimorfizm) kullanıldığında, metot çağrısının hangi sınıfa ait olduğunun çalışma zamanında belirlenmesi gerekir (dynamic dispatch). Bu, derleyici optimizasyonlarını (Örn: inline etme) kısıtlayabilir ve saf sanal olmayan metot çağrılarından biraz daha yavaş olabilir, ancak modern VM'lerde fark ihmal edilebilir düzeydedir.



Eğer bir ana sınıf, yönetilmesi gereken harici kaynaklara (dosyalar, ağ bağlantıları) sahipse, alt sınıfların da bu kaynakları düzgün bir şekilde serbest bırakması (cleanup) gerekir.

Bu, yapıcı ve yıkıcı (destructor) zincirlerinin ve 'try-finally' veya 'using' bloklarının doğru kullanılmasını zorunlu kılar.

Aksi takdirde, alt sınıflar ana sınıfın kaynaklarını açık bırakarak sızıntılara neden olabilir.



Doğru kullanıldığında:

- 1. Yeniden kullanılabilirlik ve Modülerlik artar.**
- 2. Kod hacmi azalır.**
- 3. Sınıf hiyerarşisi mantıksal bir düzen kazanır.**

Yanlış kullanıldığında (LSP ihlali veya gereksiz derinlik):

- 1. Kodun esnekliği azalır (Sıkı Bağlanma).**
- 2. Sistem bakımı zorlaşır (Kırılgan Taban Sınıfı).**

GENEL ÖZET: KALITIMIN TEMEL İŞLEVLERİ

Kalıtım, temel olarak iki işlevi yerine getirir:

1. **Özellik Aktarımı:** Ana sınıftan nitelik ve metotları alt sınıflara geçirerek kod tekrarını önler.
2. **Polimorfizm için Temel Sağlama:** Metot ezme ve ana sınıf referansları aracılığıyla esnek davranışlar oluşturulmasını sağlar.

SONUÇ: İYİ TASARIM İÇİN İPUÇLARI

1. Sadece 'is-a' ilişkisi varsa kalıtımı kullanın.
2. Kalıtım hiyerarşilerini sıkı tutun (2-3 seviyeden daha derin olmamalı).
3. Ana sınıfı (Superclass) çok genel tutmaktan kaçının (çok fazla sorumluluk yüklemeyin).
4. Mümkünse, somut sınıflardan kalıtım yerine arayüzleri uygulamayı veya soyut sınıfları kullanmayı tercih edin.
5. Çoğu durumda, Kompozisyonu Kalıtıma tercih etmeyi değerlendirin (Has-a ilişkileri).

KALITIM

Süleyman EZDEMİR

suleyman.ezdemir@giresun.edu.tr